

Exercise Sheet 1: Sparse Information Retrieval

Relevant code: IR: CO5 repo LINK

This exercise focuses on building a **mini search engine** to retrieve the most relevant abstracts from **MedLine** given a user query. **MedLine** is a collection of more than 30 million biomedical scientific papers' abstract in life science and medicine. You will work through data acquisition, indexing, querying, and (optionally) evaluating performance using MeSH-term prediction.

Overview

You will:

1. Download the MedLine dataset in bulk.
 2. Index abstracts using the **Whoosh** search library.
 3. Perform full-text search with various query logics.
 4. *(Optional)* Evaluate system performance using MeSH-term prediction.
-

Step 1 — Download MedLine in Bulk

Retrieve the MedLine dataset using the official **NCBI FTP repository**. This provides large-scale access to medical abstracts required for indexing. This is already done for you in **I_fetch_pubmed.py**. Make sure you understand the code before committing to the next step. Pay special attention to line 65 for time management.

Step 2 — Index the MedLine Data Using Whoosh

Use the Whoosh library for indexing. Read the documentation and use the sample code for indexing your data.

Whoosh documentation (indexing):

<https://whoosh.readthedocs.io/en/latest/indexing.html>

Index at least the following fields:

- **Title**
- **Body (Abstract)**
- *(Optional)* **MeSH terms**

This will allow full-text search across medically relevant fields.

Step 3 — Search the Indexed Data

Use Whoosh's query system to execute searches:

Whoosh documentation (searching):

<https://whoosh.readthedocs.io/en/latest/searching.html>

Experiment with different query semantics:

- **OR queries**
- **AND queries**
- **Phrase queries**
- **Fuzzy vs. Exact match**
- **Different scoring methods**

Subjectively evaluate how these choices influence retrieval quality.

Step 4 (Optional) — MeSH-Term Prediction and Evaluation

This optional step introduces a simple evaluation pipeline using MeSH terms.

4.1 Build Training and Test Datasets

1. **Sample a large set of abstracts** (e.g., 1,000,000)
 - Create a mapping:
PMID → [**Title** + **Abstract**, [**MeSH terms**]]
 - This becomes your **training dataset**.
2. **Sample a smaller set** (e.g., 1,000 abstracts)
 - This becomes your **test dataset**.

4.2 Build an Index Over MeSH Terms

Using the training dataset:

- Index each **Title** + **Abstract** string. **Ignore MeSH-Terms** as otherwise it makes the task too easy and unrealistic.
- Searching a MeSH term should return the PMIDs associated with it.

4.3 Evaluate Prediction Accuracy

For each PMID in the **test dataset**:

1. Treat the MeSH-terms of each PMID as a sentence and query them by the index.
2. Among the hit PMIDs take the one with **the highest score**.
3. Compare it to the original test PMID by conducting following mettrics.

Metrics

- **Accuracy@1**
Proportion of cases where the correct PMID is the **top hit**.
- **Accuracy@5**
Proportion of cases where the correct PMID appears within the **top 5 hits**.

Evaluation Questions

- What do these accuracy values reveal about your system performance?
 - How do indexing choices and query formulation affect results?
 - What improvements could be made?
-

Congratulations

You just developed a mini version of PubMed, the most popular search engine in the medical and life science community!